# PRD → AI Coding Tool Files Generator: Complete Reference

## Service Concept

A service that takes a **PRD (Product Requirements Document)** as input and generates **all the configuration/context files** needed by the 4 major AI coding tools — so developers can immediately start building with any tool of their choice.

---

## Summary: What Each Tool Needs

| Tool | Primary Config File | Config Directory | Skills/Extensions | Additional Files |
|---|---|---|---|---|
| **Claude Code** | `CLAUDE.md` | `.claude/` | `.claude/agents/`, Skills via `SKILL.md` | `.claude/settings.json`, `.claude/settings.local.json` |
| **Cursor** | `.cursor/rules/*.mdc` + `AGENTS.md` | `.cursor/` | `.cursor/rules/`, Skills via `SKILL.md` | `.cursor/modes.json`, `.cursorrules` (legacy) |
| **OpenAI Codex** | `AGENTS.md` | `.codex/` | Skills via `/skills/` | `.codex/config.toml`, `AGENTS.override.md` |
| **Google Antigravity** | `.agent/rules/` + `AGENTS.md` | `.agent/` | `.agent/skills/` via `SKILL.md` | Rules, Workflows, `.antigravity/rules.md` |

---

## 1. Claude Code

### File Structure

```
project-root/
├── CLAUDE.md                    # Primary — auto-read at session start
├── .claude/
│   ├── settings.json            # Shared project settings (git-tracked)
│   ├── settings.local.json      # Personal settings (git-ignored)
│   ├── agents/                  # Custom AI sub-agents
│   │   └── reviewer.md          # Example: code review agent
│   └── commands/                # Slash commands
│       └── deploy.md            # Example: /deploy command
├── docs/                        # Referenced by CLAUDE.md
│   └── architecture.md
└── subdirectory/
    └── CLAUDE.md                # Nested — scoped to this dir
```

### CLAUDE.md Structure (from PRD)

```markdown
```

# Project Context

## About This Project

[Extract from PRD: product description, goals, target users]

## Tech Stack

- Frontend: [from PRD tech decisions]
- Backend: [from PRD tech decisions]
- Database: [from PRD tech decisions]
- Testing: [from PRD tech decisions]

## Key Directories

- `src/models/` - database models
- `src/api/` - route handlers
- `src/core/` - configuration and utilities

## Standards

- [From PRD: coding standards, type requirements]
- [From PRD: testing strategy]
- [From PRD: naming conventions]

## Common Commands

```bash
npm run dev        # development server
npm test           # run tests
npm run lint       # linting
npm run build      # production build
```

## Architecture Decisions

[From PRD: key architectural patterns, data flow]

## Important Warnings

```
- [From PRD: security constraints, performance requirements]
- [From PRD: do NOT modify files in X directory]
```

### .claude/settings.json

```
  json
{
 "permissions": {
  "allow": ["Bash(npm run *)", "Bash(git *)"],
  "deny": ["Read(./.env)", "Read(./.env.*)", "Read(./secrets/**)"]
 }
}
```

### Key Principles

- **Less is more** — keep CLAUDE.md concise (acts as system prompt)
- **Progressive disclosure** — tell Claude *how to find* info, not everything
- **Evolve over time** — update when you notice friction
- **Hierarchy**: `~/.claude/CLAUDE.md` (global) → `project/CLAUDE.md` → `project/sub/CLAUDE.md`

---

## 2. Cursor

### File Structure

```
project-root/
├── AGENTS.md                    # Universal agent instructions (also read by Cursor)
├── .cursor/
│   ├── rules/
│   │   ├── project-overview.mdc      # Always applied (alwaysApply: true)
│   │   ├── frontend-patterns.mdc     # Auto-attached for frontend files
│   │   ├── backend-patterns.mdc      # Auto-attached for backend files
```

```
│   │   ├── testing.mdc          # Auto-attached for test files
│   │   └── deployment.mdc       # Agent-requested (on demand)
│   ├── modes.json               # Custom agent modes
│   └── hooks/
│       └── grind.ts             # Post-action hooks
├── .cursorrules                 # Legacy (still supported, deprecated)
└── subdirectory/
    └── AGENTS.md                # Nested agent instructions
```

### .mdc Rule File Format

```markdown
```

```
---
description: React component patterns for the frontend
globs: src/components/**/*.tsx, src/components/**/*.ts
alwaysApply: false
---

# Frontend Component Rules

## Tech Stack
- React 18 + TypeScript
- Tailwind CSS for styling
- React Query for data fetching

## Patterns
- Use functional components with hooks
- Always include error boundaries
- Props must be typed with interfaces (not type aliases)

## File Structure
- Component: `ComponentName.tsx`
- Test: `ComponentName.test.tsx`
- Styles: Use Tailwind classes inline
```

### Rule Types (inferred from frontmatter)

| Rule Type | How It Works | Frontmatter |
|---|---|---|
| **Always** | Always in context | `alwaysApply: true` |
| **Auto-Attach** | When matching files are referenced | `globs: "src/**/*.tsx"` |
| **Agent** | Agent decides when relevant | `description: "..."` only |
| **Manual** | Only when explicitly referenced | No description, globs, or alwaysApply |

### What to Generate from PRD

| PRD Section | → Cursor File |
|---|---|
| Project overview, tech stack | `project-overview.mdc` (alwaysApply: true) |
| Frontend architecture | `frontend-patterns.mdc` (globs: `src/components/**`) |
| Backend architecture | `backend-patterns.mdc` (globs: `src/api/**`, `src/services/**`) |
| Testing strategy | `testing.mdc` (globs: `**/*.test.*`, `**/*.spec.*`) |
| Database/models | `database.mdc` (globs: `src/models/**`, `prisma/**`) |
| Deployment/CI/CD | `deployment.mdc` (agent-requested) |
| API design | `api-design.mdc` (globs: `src/api/**`) |
| Security requirements | `security.mdc` (alwaysApply: true) |

## 3. OpenAI Codex

### File Structure

```
~/.codex/
├── AGENTS.md                # Global defaults (all projects)
├── AGENTS.override.md           # Temporary global override
└── config.toml              # CLI configuration

project-root/
├── AGENTS.md                # Project-level instructions
```

```
├── .codex/
│   └── config.toml                # Project-level config
├── packages/
│   └── payments/
│       ├── AGENTS.md              # Module-specific instructions
│       └── AGENTS.override.md     # Module override
```

### AGENTS.md Structure (from PRD)

```markdown
```

```
# Project: [Name from PRD]

## Overview
[Product description from PRD]

## Working Agreements
- Always run `npm test` after modifying JavaScript files
- Prefer `pnpm` when installing dependencies
- Ask for confirmation before adding new production dependencies

## Architecture
[From PRD: system architecture, patterns]

## Build & Test Commands
- `pnpm dev` - Start development server
- `pnpm test` - Run all tests
- `pnpm lint` - Lint code
- `pnpm build` - Production build

## Code Standards
[From PRD: coding conventions, patterns to follow]

## Security
[From PRD: security constraints, sensitive files to avoid]

## Deployment
[From PRD: deployment process, environments]
```

### .codex/config.toml

```toml
```

```toml
model = "gpt-5-codex"
approval_policy = "on-failure"
sandbox_mode = "workspace-write"

[features]
shell_snapshot = true

[shell_environment_policy]
include_only = ["PATH", "HOME", "NODE_ENV"]
```

### Discovery Hierarchy (highest → lowest precedence)

1. `~/.codex/AGENTS.override.md` (global override)
2. `~/.codex/AGENTS.md` (global defaults)
3. `project-root/AGENTS.md` → walk down to current dir
4. Each directory: `AGENTS.override.md` → `AGENTS.md` → fallback filenames

### Key Details

- **Max file size**: 32KB default (`project_doc_max_bytes`)
- **Fallback filenames**: configurable in `config.toml`
- **Merge order**: root → down, later files override earlier
- Supports MCP servers via `config.toml`

---

## 4. Google Antigravity

### File Structure

```
~/.gemini/antigravity/
└── skills/                    # Global skills (all projects)
```

```
  └── code-review/
      └── SKILL.md

project-root/
├── AGENTS.md                    # Universal agent instructions
├── .agent/
│   ├── rules/                   # Project rules
│   │   ├── coding-standards.md
│   │   ├── architecture.md
│   │   └── security.md
│   ├── skills/                  # Project-specific skills
│   │   ├── deploy-staging/
│   │   │   └── SKILL.md
│   │   └── generate-tests/
│   │       ├── SKILL.md
│   │       └── scripts/
│   │           └── generate.py
│   └── workflows/               # Automated workflows
│       └── generate-tests.yaml
├── .antigravity/
│   └── rules.md                 # Alternative rules location
```

### Rules (Always-On Context)

Rules are always loaded into the agent's context. Defined via IDE UI or file-based.

```markdown
```

```
# Coding Standards Rule

## Language
- Use TypeScript for all new files
- Strict mode enabled
- No `any` types

## Architecture
- Follow Controller → Service → Repository pattern
- All API responses use standard envelope format

## Testing
- Every new feature requires unit tests
- Integration tests for API endpoints
- Use Vitest for testing
```

### Skills (On-Demand, Agent-Triggered)

Skills are loaded only when relevant. Directory-based with `SKILL.md`.

```markdown
```

```
# SKILL.md — Deploy to Staging

## Description
Deploy the current branch to the staging environment. Use this when
the developer asks to "deploy", "push to staging", or "test in staging".

## Prerequisites
- All tests must pass before deployment
- Branch must be up to date with main

## Steps
1. Run `npm run build`
2. Run `npm test`
3. Execute `./scripts/deploy-staging.sh`
4. Verify deployment at https://staging.example.com

## Verification
- Check that the health endpoint returns 200
- Run smoke tests via `npm run test:smoke`
```

### Workflows (Command-Triggered, `/command`)

```yaml
```

```yaml
# .agent/workflows/generate-tests.yaml
name: Generate Tests
description: Generate unit tests for modified files
trigger: /generate-tests
steps:
  - name: Find modified files
    command: git diff --name-only HEAD~1
  - name: Generate tests
    prompt: "Write comprehensive unit tests for each modified file"
  - name: Run tests
    command: npm test
```

### Key Concepts

- **Rules** = Always loaded (like system prompt) — for coding standards, architecture
- **Skills** = On-demand (agent decides when relevant) — for specific tasks, scripts
- **Workflows** = User-triggered via `/command` — for multi-step automation
- **Artifacts** = Agent-generated deliverables (plans, screenshots, recordings)
- **Manager View** = Multi-agent orchestration (parallel tasks)

---

## 5. Universal: AGENTS.md (Cross-Tool Standard)

AGENTS.md is emerging as the **universal standard** supported by all major tools:

| Tool | AGENTS.md Support |
| --- | --- |
| OpenAI Codex | ✅ Native (primary file) |
| Cursor | ✅ Native support |
| Google Antigravity / Gemini | ✅ Supported |
| VS Code Copilot | ✅ Supported |
| Claude Code | ⚠️ Workaround: `ln -s AGENTS.md CLAUDE.md` |

**Stewarded by**: Agentic AI Foundation under the Linux Foundation

### Recommended AGENTS.md Template (from PRD)

```markdown
```

# [Project Name]

## Overview

[From PRD: What the product does, who it's for]

## Tech Stack

[From PRD: Languages, frameworks, databases, infrastructure]

## Architecture

[From PRD: System design, key patterns, data flow]

## Directory Structure

[From PRD: Key directories and their purposes]

## Development Commands

```bash
# Setup
npm install

# Development
npm run dev

# Testing
npm test                    # all tests
npm test -- --watch         # watch mode
npm run test:e2e            # e2e tests

# Build & Deploy
npm run build
npm run deploy:staging
```

## Code Standards

[From PRD: Naming conventions, patterns, linting rules]

## **Testing Requirements**

[From PRD: Test coverage, testing strategy, test patterns]

## **Security Constraints**

[From PRD: Auth patterns, sensitive files, API key handling]

## **Known Issues / Gotchas**

[From PRD: Edge cases, workarounds, technical debt]

---

## 6. PRD-to-Files Mapping Matrix

This is the core logic of your service — mapping PRD sections to output files:

| PRD Section | CLAUDE.md | Cursor .mdc | Codex AGENTS.md | Antigravity |
|---|---|---|---|---|
| Product Description | ✅ About section | ✅ project-overview.mdc | ✅ Overview | ✅ Rules |
| Tech Stack | ✅ Tech Stack | ✅ project-overview.mdc | ✅ Tech section | ✅ Rules |
| Architecture | ✅ Architecture | ✅ architecture.mdc (agent) | ✅ Architecture | ✅ Rules + Skills |
| Frontend Specs | ✅ Standards | ✅ frontend.mdc (auto-attach) | ✅ nested AGENTS.md | ✅ Skills |
| Backend Specs | ✅ Standards | ✅ backend.mdc (auto-attach) | ✅ nested AGENTS.md | ✅ Skills |
| API Design | ✅ Important Notes | ✅ api.mdc (auto-attach) | ✅ Code Standards | ✅ Rules |
| Database Schema | ✅ Important Notes | ✅ database.mdc (auto-attach) | ✅ nested AGENTS.md | ✅ Skills |
| Testing Strategy | ✅ Commands + Standards | ✅ testing.mdc (auto-attach) | ✅ Working Agreements | ✅ Workflows |
| Security Reqs | ✅ Warnings | ✅ security.mdc (always) | ✅ Security section | ✅ Rules (always) |
| Deployment | ✅ Commands | ✅ deployment.mdc (agent) | ✅ Deployment | ✅ Workflows |
| UI/UX Specs | ✅ Important Notes | ✅ ui-patterns.mdc | ✅ in AGENTS.md | ✅ Skills |
| Error Handling | ✅ Standards | ✅ error-handling.mdc | ✅ Code Standards | ✅ Rules |

| PRD Section | CLAUDE.md | Cursor .mdc | Codex AGENTS.md | Antigravity |
|---|---|---|---|---|
| Performance Reqs | ✅ Warnings | ✅ performance.mdc | ✅ in AGENTS.md | ✅ Rules |
| 3rd Party APIs | ✅ Important Notes | ✅ integrations.mdc | ✅ nested AGENTS.md | ✅ Skills |

## 7. Output File Bundle Structure

When your service processes a PRD, it should generate a downloadable ZIP with:

```
output-bundle/
├── README.md                    # How to use these files
├── AGENTS.md                    # Universal (works everywhere)
│
├── claude-code/
│   ├── CLAUDE.md
│   └── .claude/
│       └── settings.json
│
├── cursor/
│   ├── .cursor/
│   │   └── rules/
│   │       ├── project-overview.mdc   # alwaysApply: true
│   │       ├── frontend.mdc           # auto-attach: src/components/**
│   │       ├── backend.mdc            # auto-attach: src/api/**
│   │       ├── testing.mdc            # auto-attach: **/*.test.*
│   │       ├── security.mdc           # alwaysApply: true
│   │       └── deployment.mdc         # agent-requested
│   └── .cursorrules                   # Legacy fallback
│
├── codex/
│   ├── AGENTS.md                      # (symlink or copy of root)
│   └── .codex/
│       └── config.toml
│
└── antigravity/
    ├── .agent/
    │   ├── rules/
    │   │   ├── coding-standards.md
    │   │   ├── architecture.md
    │   │   └── security.md
    │   ├── skills/
    │   │   └── [generated skills]/
```

```
│   │       └── SKILL.md
│   └── workflows/
│       └── [generated workflows].yaml
└── .antigravity/
    └── rules.md
```

---

## 8. Key Design Principles for Your Service

### Content Optimization Per Tool

| Principle | Claude Code | Cursor | Codex | Antigravity |
|---|---|---|---|---|
| **Max context** | Keep small (<500 lines) | Split into many .mdc files | 32KB per file | Rules small, Skills on-demand |
| **Hierarchy** | Nested CLAUDE.md | globs + alwaysApply | Nested AGENTS.md | Rules vs Skills vs Workflows |
| **Style** | Concise bullet points | Structured with frontmatter | Markdown sections | Markdown + YAML workflows |
| **Commands** | In CLAUDE.md directly | In always-apply rule | In AGENTS.md | In Rules or Workflows |
| **Security** | In settings.json deny list | In always-apply rule | In AGENTS.md + config.toml | In Rules (always-on) |

### PRD Fields Your Service Should Extract

1. **Product Identity**: name, description, goals, target users
2. **Tech Stack**: languages, frameworks, databases, infrastructure, CI/CD

3. **Architecture**: patterns (MVC, microservices), data flow, key modules
4. **Directory Structure**: source layout, key directories, file conventions
5. **Development Commands**: setup, dev server, test, build, deploy, lint
6. **Code Standards**: naming, formatting, typing rules, patterns
7. **Testing Strategy**: frameworks, coverage targets, test types
8. **Security Requirements**: auth, secrets handling, API keys, RBAC
9. **API Design**: REST/GraphQL patterns, error formats, versioning
10. **Database Design**: schema, migrations, ORM patterns
11. **UI/UX Patterns**: component library, design system, responsive rules
12. **Deployment**: environments, CI/CD pipeline, rollback procedures
13. **3rd Party Integrations**: external APIs, SDKs, services
14. **Performance Requirements**: load times, caching, optimization rules
15. **Error Handling**: patterns, logging, monitoring, alerting

---

## 9. Monetization & Positioning Ideas

- **Free tier**: Generate AGENTS.md only (universal)
- **Pro tier**: All 4 tool bundles + customization
- **Team tier**: Shared rules synced across repos + team conventions
- **Integration**: Clearly PRD → one-click export to all tools
- **Marketplace**: Pre-built templates per framework (Next.js, FastAPI, Flutter, etc.)